

Hands On Module 2

Artificial Intelligence

Hands-On Task

Task Overview	A comprehensive analysis of the vehicle fuel efficiency (MPG) dataset using NumPy and Seaborn/Matplotlib. You will apply statistics, linear algebra, EDA, and visualisation concepts to answer concrete engineering questions about the factors that influence fuel efficiency.
Dataset	MPG Dataset from Seaborn: <code>df = sns.load_dataset('mpg')</code>
Tools Required	Python (Jupyter/Colab), NumPy, Pandas, Seaborn, Matplotlib
Expected Output	A Notebook (.ipynb) with all code running, all plots visible, and markdown cells explaining every finding. The notebook must run top-to-bottom without errors.
Portfolio Alignment	This Hands-On Task aligns participants portfolios with professional AI industry standards by producing an end-to-end data science project that showcases core competencies in exploratory data analysis (EDA), statistical visualization, and coding advanced mathematical foundations from scratch like the Normal Equation and PCA.
Assessment Details	
Code correctness, quality of analytical interpretation, visualisation completeness (titles, labels, legends), and depth of markdown explanations	

Task Specification

Stage 1 Inspection and Data Cleaning

1. Load the dataset and display its shape, dtypes, and missing values per column. Identify which columns have missing values and handle them with an appropriate strategy provide a written justification for your choice (drop row vs imputation), not just the code.
2. Build a summary table using NumPy (without Pandas .describe()) that shows mean, median, std, IQR, and outlier count (IQR $\times 1.5$ method) for every numeric column. Format the output as a clean DataFrame.

Stage 2 Statistical Analysis with NumPy

3. Extract the numeric features ['mpg', 'displacement', 'horsepower', 'weight', 'acceleration'] as a NumPy array. Apply z-score standardisation in a vectorised manner (no sklearn, no loops).
4. Compute the correlation matrix of those 5 features using `np.corrcoef()`. Identify: (a) the feature pair with the strongest positive correlation, (b) the feature pair with the strongest negative correlation to mpg, and (c) the input feature pair most correlated with each other (potential multicollinearity).
5. Use NumPy boolean masking to answer: do cars with above-average horsepower ($>$ mean horsepower) also tend to weigh more than the dataset average? State the absolute difference and interpret it.

Stage 3 Visualisation (Minimum 4 Plots)

6. MPG Distribution: histogram with KDE overlay, vertical lines marking mean and median with annotated values. The plot must answer: is the mpg distribution symmetric or skewed?
7. Origin Comparison: box plot or violin plot showing the mpg distribution per origin (USA, Europe, Japan). Include an interpretation: which country produces the most fuel-efficient vehicles, and how consistent are the results?
8. Weight vs MPG Relationship: scatter plot with hue by cylinders, a manual trend line using `np.polyfit()`, and the correlation value displayed in the title.
9. Correlation Heatmap: visualise the correlation matrix from Stage 2. Use `annot=True` and a meaningful diverging colormap. Identify and name any multicollinearity patterns visible in the heatmap.

Stage 4 Contextual Interpretation

10. Based on your entire analysis, answer in a markdown cell: What factor most strongly predicts a vehicle's fuel efficiency? support your answer with correlation values, boxplot observations, and any other findings from your analysis.
11. Display the mean mpg per decade (`model_year` rounded to the nearest ten) using Pandas groupby, then plot it as a line chart showing the temporal trend. Interpret: why might this trend be occurring?

Bonus 1 – Normal Equation from Scratch

Implement Linear Regression without sklearn using the Normal Equation:

$$\hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Use weight as the single input feature X to predict mpg as y . Steps:

1. Add a bias column (column of 1s) to X , forming matrix $\mathbf{X} \in \mathbb{R}^{n \times 2}$
2. Compute $\hat{\theta}$ using `np.linalg.inv()` or `np.linalg.lstsq()`

3. Plot the resulting regression line over a scatter plot of the raw data
4. Compute RMSE: $\sqrt{\frac{1}{n} \sum (\hat{y}_i - y_i)^2}$
5. Compare your result with `np.polyfit()` – are the slope and intercept the same?

Bonus 2 – PCA from Scratch

Implement 2D Principal Component Analysis (PCA) using only NumPy to visualise the structure of the 5 numeric features in two dimensions:

1. Mean-centre all features: $\tilde{\mathbf{X}} = \mathbf{X} - \boldsymbol{\mu}$
2. Compute the covariance matrix: $\boldsymbol{\Sigma} = \frac{1}{n-1} \tilde{\mathbf{X}}^T \tilde{\mathbf{X}}$
3. Compute eigenvalues and eigenvectors using `np.linalg.eigh()`
4. Select the 2 eigenvectors with the largest eigenvalues (principal components)
5. Project the data to 2D: $\mathbf{Z} = \tilde{\mathbf{X}} \mathbf{V}_2$
6. Plot the 2D scatter with hue by origin, and compute the explained variance ratio for PC1 and PC2

Bonus 3 – Reusable EDA Class

Build a `DatasetProfiler` class with at least 5 methods usable for EDA on any numeric dataset:

- `__init__(self, df, target_col)` – store the dataframe and target column name
- `summary_stats()` – return a `DataFrame` of descriptive statistics with IQR and outlier count via NumPy
- `correlation_report()` – return a `Series` of all features' correlations to the target, sorted
- `plot_dashboard()` – generate a figure with 4 subplots in one call: target distribution, boxplot by category (if available), correlation heatmap, scatter of strongest feature vs target

- `generate_report()` → `dict` – return a JSON-serialisable dictionary of all key findings

Demonstrate the class on the MPG dataset (`target_col='mpg'`) with all methods callable from a single cell.